

Tema 3: Llistes i iteradors

3.1 Llistes vs vectors

Les **l·listes** (**list**) solucionen l'alt cost d'inserció al mig dels vectors $\mathcal{O}(n)$. Estan formades per nodes independents enllaçats. Afegir o esborrar un element intermig costa només $\mathcal{O}(1)$.

En algorísmia, $\mathcal{O}(n)$ (es pronuncia “O de n”) significa que **el temps o cost d'execució creix de manera lineal** a mesura que entren més dades. Per exemple: `cout << "Hello World" << endl;` és $\mathcal{O}(1)$, un element constant. Una pitjor: $\mathcal{O}(n^2)$ `for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { ... } }`.

Desavantatges algorísmics: - **Sense posicions directes:** Utilitzar `L[i]` genera error de compilació. - **Cost de travessia:** Per arribar a n , cal recórrer seqüencialment tots els nodes anteriors.

Mètodes de Llistes ($\mathcal{O}(1)$ garantit): `push_back()`, `push_front()`, `pop_back()`, `pop_front()`, `front()` i `back()`.

Encara que les llistes siguin de cost constant en mig de la seqüència, en termes generals d'eficiència s'acostuma a prioritzar l'ús del `std::vector` atès que emmagatzema memòria en blocs contigus molt llestos de llegir. Només usarem llistes si el problema exigeix constants insercions i esborrats intermedis.

3.2 Iteradors

Davant la manca d'índexs numèrics (com `[i]`), les llistes s'han de recórrer usant **Iteradors**. L'iterador funciona de factor formal com a un punter tàctic d'aquell element actiu:

- `L.begin()`: Retorna l'iterador apuntant al **primer** element.
- `L.end()`: Retorna l'iterador que assenyala la cel·la virtual **després de l'últim** element (fora de rang).
- S'accedeix al seu valor utilitzant l'asterisc com a desreferenciació: `*it = 50`.
- Es passa al següent element avaluant el símbol suma: `it++`.

```
list<int> L = {10, 20, 30};
```

```
// S'usa 'auto' per simplificar tipus extremadament llargs com 'list<int>::iterator'
for (auto it = L.begin(); it != L.end(); it++) {
    *it += 5;
}
```

Variants principals d'iteradors: - **const_iterator** (`cbegin`, `cend`): Si la llista es passa com a constant `const`, no permet mutar les dades mitjançant `*it = x;`. - **reverse_iterator** (`rbegin`, `rend`): Permet recórrer la llista del final al principi mantenint comoditat tècnica aplicant en el fons normalment el `it++`.

Retrocedir manualment des de `L.end()` amb iteradors porta problemes tècnics d'índex ja que l'avaluació arrenca “al límit on ja no hi ha res”. Observa de quina manera avança el simulador respecte al traçat invers:

[Simulació interactiva disponible a la web]

3.3 El perill d'alterar l'itinerari avançat: Insercions

Esborrar o afegir un element on tenim actualment ancorat el punter a la meitat d'una seqüència generarà pràcticament la pèrdua d'orientació interna llançant un *Segmentation Fault*: l'adreça activa anterior ha quedat completament alienada i `it++` ja no sap a quin objecte “següent” enllaçar.

Per això en un ús d'enginyeria, C++ retorna **un nou iterador ja enfocat en localització lícita** següent quan uses:

- `it = L.insert(it, valor)`: Insereix **abans** de la posició i el fixa al punt original.
- `it = L.erase(it)`: Esborra element i el fixa sobre l'element a la dreta que ocuparà actualment aquest buit.

El protocol per gestionar-ho correctament exigeix evitar for loops basant-se en declaracions per patró `while`:

```
void netejar_llista(list<int>& L) {
    auto it = L.begin();

    while (it != L.end()) {
        if (*it == 10) {
            it = L.erase(it);    // Salvem de l'oblit el desvincular! Torna el següent
        }
        else if (*it == -1) {
            it = L.insert(it, 0);
            advance(it, 2);      // Avancem l'enfocat fora del radi de read paper de la memòria
        }
        else {
            it++;                // Pas d'iteració ordinària natural
        }
    }
}
```

Aquest fenomen no és únic. Utilitzar i recórrer amb `std::vector` està sotmès sota els mateixos efectes destructius pel sistema si elimines valors usant `vector.erase(it)` i intentes fer `it++` corrent cecament seguidament a C++.

Visualitza pas a pas en primera persona al projecte l'assecurament tècnic iterador observant quins rols tornen per reengantjar al segle!

[Simulació interactiva disponible a la web]